

ML Specification Report — tabular-playground-series-jan-2022

Forecast Daily Kaggle-Store Sales · our from-scratch agent (structural Ridge + tree-search v3, honest-gated solo)

Figures grounded in the competition's `STATUS.md`, `config.yaml`, `dossier.json`, `experiments.json`, `experiments_tree_v3.json`, `final_decision.json`, and `headless_run.log`, with leaderboard figures from the study's frozen three-way score table. Version discipline: this competition ran 2026-07-28 under the pre-v5 (v3-frozen) pipeline, the same configuration as the rest of the frozen 18-competition baseline — Stage 0.5 did not yet exist. The cited `dossier.json` was produced by the retrospective problem-identification sweep of 2026-07-29/30 and was NOT consumed by this run; it is cited only as a descriptive record of the task.

Contents

• Forecast Daily Kaggle-Store Sales · our from-scratch agent (structural Ridge + tree-search v3, honest-gated solo)	1
• Overview	1
• Data	2
• Models & Architecture	3
• Training procedures	3
• Inference procedures	5
• Evaluation & Benchmarking	5
• Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08	6

Overview

The task is to forecast daily unit sales (`num_sold`) of Kaggle merchandise for all of 2019, given 2015–2018 history across 18 (country × store × product) series (metric: SMAPE, minimize). We solve it with a from-scratch pipeline whose champion is deliberately *not* a blend: a **structural Ridge regression on log-sales** — a World Bank GDP-per-capita level term, linear year trend, day-of-week and Fourier × product seasonality, and per-holiday-name dummies — scored on honest expanding-year folds at **CV SMAPE 4.415985**. A 60-node tree search then surfaced an apparent 37-member mega-blend at OOF 4.40502, but a leave-year-out honest gate exposed that gain as +0.032 of fitted-weight optimism, so the root solo was submitted. On the real leaderboard the submission scores **Public 4.90883 / Private 6.46156**, rank **455/1592** (percentile 71.5): the best *local* CV of the three benchmarked agents (`local_winner` = mine), but the **NVIDIA reproduce-agent wins on the leaderboard** (Private 4.63651, percentile 76.4), with AIDE far behind (Private 10.66638, percentile 28.9).

Why it matters. Multi-series retail demand forecasting — level, trend, seasonality, and holiday effects per country — is the canonical inventory-and-revenue planning problem; this episode additionally tests whether

an agent can exploit *permitted* external data (macro indicators, holiday calendars) and resist overfit blends when the test window lies strictly in the future.

Data

Purpose of Data. Forecast the number of units sold per day for every (date × country × store × product) series — a tabular time-series panel from the Tabular Playground family. **Data Format** is clean **tabular CSV** with four raw predictive columns (date , country , store , product) once row_id and the target num_sold are removed. **Data Volume** is **26,298 training rows (2015-01-01 → 2018-12-31, 1,461 days × 18 series) / 6,570 test rows (all of 2019, 365 × 18)** — a complete panel with no gaps or duplicate keys, and the test set contains exactly the same 18 series (3 countries × 2 stores × 3 products; no cold-start).

Data Quality is high: **0 nulls, 0 duplicate keys**, target an integer count with **no zeros** (min 70, max 2884, mean 387.5). Yearly mean num_sold runs 364.4 / 364.8 / 396.0 / 425.0 for 2015–2018 — a growing trend the model must extrapolate blind into 2019. The strongest EDA signals, all re-confirmed on data from the prior-run experience notes, point to a rigidly multiplicative structure:

- **Store ratio is constant** (KaggleRama / KaggleMart = 1.742 in every year); **product shares are constant** across years *and* countries; the **day-of-week effect is identical** across stores and countries.
- **total(country, year) / gdp_pc is nearly equal across countries** (14.22 / 14.48 / 14.87 / 14.96) — only a common year drift remains, which is exactly what a log_gdp + linear-year term absorbs; sharp daily spikes align with national holidays, motivating per-holiday-name dummies rather than a single flag.

Annotation Guidelines. The label is num_sold ∈ ℤ⁺ (integer daily units). Submissions are sales forecasts scored by **SMAPE** = (100/n) · Σ 2|F-A| / (|A|+|F|) (percent convention; the pipeline's metric function is the equivalent 200 × mean form) — a scale-free, MAE-family regression error where every series counts equally per row.

Feature Set. External data is explicitly allowed (config.yaml special rules), and the design matrix leans on it — **119 columns** for the CV champion (experiments.json exp #3; the final-refit entry logs n_features = 101 — the two log entries disagree and are reported as logged):

Group	Features
Level & trend	log_gdp (World Bank NY.GDP.PCAP.CD per country × year, fetched live incl. 2019), year_c (centered year)
Entity effects	store dummy (KaggleRama), product dummies (Mug / Sticker)
Weekly	6 day-of-week dummies
Yearly seasonality	Fourier(8) on day-of-year + Fourier(8) × product interactions
Holidays	per-holiday-name dummies (English names via the holidays package, pooled across FI/NO/SE), shift window -5..+10 days

Splitting strategy. The test year (2019) lies strictly *after* the training span, so validation is **expanding-year folds**: $\text{train} \leq 2016 \rightarrow \text{validate } 2017$, $\text{train} \leq 2017 \rightarrow \text{validate } 2018$, pooled — each fold a *full calendar year* so the seasonal mix matches the test year, with **round-to-int applied inside the metric** so every candidate is scored on its post-processed output. Shuffled KFold is explicitly forbidden by the problem dossier (it leaks future days of the same series into training). The folds are deterministic (no fold seed needed). During the 2026-07-28 run the scores were **OOF-only** (competition closed; API late submission typically rejected for pre-2022 playgrounds); the public/private figures in this report come from the study's later frozen three-way scoring of the produced `submission.csv` against the final leaderboard.

Models & Architecture

Purpose of Architecture. A daily-sales forecaster that minimizes SMAPE under a one-year-ahead extrapolation. **Architecture Type** is — unusually for this report set — a **penalized linear structural model**: Ridge regression ($\alpha = 0.1$, closed-form) on `log(num_sold)`, chosen because the panel's multiplicative structure is near-exact and the GBDT alternative cannot extrapolate the 2019 level. The champion is the **root solo** `ridge_hol`; every fitted blend the tree search proposed was vetoed by the honest gate. Each model takes the same **Input Format** — a float design matrix built from the four raw columns plus the two external joins — of **Input Dimension 119 columns** per row (a 1-D vector; temporal structure is encoded as trend, Fourier, and holiday-shift features).

Architecture Description. The structural model is `log(num_sold) ~ log_gdp + year_c + store + product + dow + Fourier(8)×product + per-holiday-name shifted dummies`, solved in closed form; predictions are exponentiated back, rounded, and clipped. **Model Complexity** is a single linear coefficient vector over the 119 columns — the smallest champion in this benchmark set. The pool's diversity member is `lgb_gdp`: LightGBM (`num_leaves` = 31, `learning_rate` = 0.05, `n_estimators` = 800) on a `log(num_sold/gdp_pc)` target.

Training procedures

Training proceeds as a **baseline ladder**, then a **tree-search layer**, then an **honesty gate**, all on the same pooled expanding-year OOF scale:

Stage	Configuration	Pooled SMAPE
ridge_base	Ridge, Fourier(4), no holidays	10.806697
ridge_fp	+ Fourier(8) × product	7.021504
ridge_hol	+ per-holiday-name dummies (−5..+10)	4.415985
lgb_gdp	LightGBM on <code>log(num_sold/gdp_pc)</code> ; grid blend with <code>ridge_hol</code> collapses to weight 1.0 ridge (4.415985)	6.329287
tree search v3	node #59, 37-member mega-blend (full-OOF)	4.40502 — <i>vetoed</i>

The **tree search** (harness_v3 , driver run_tpsjan22_v3.py) evaluated **60/60 nodes (44 solo / 16 blend) in 91 s wall-clock**, root digit-verified at 4.415985, across 9 lineages (ALPHA, FOURK, HOLWIN, PERPROD, GDPOFF, DOWPROD, RECENT, LGBDIV, BLEND) plus a 5-seed explore burst. **No structural mutation beat the root solo** (best solo child 4.417218 in the ALPHA lineage per STATUS.md ; the tree log's lowest non-root solo, a FOURK α -nudge at 4.416732, likewise loses) — every apparent gain was a fitted-weight blend. Bookkeeping: evals-to-match-linear-best = 1 (the root is the linear champion), 1 dedup rejection, cost-guard fired 0 times, the burst sanity gate killed EXPL_COMBO (4.690) and EXPL_LGBBIG (6.429), 16 backtrack events, stop reason = hard budget cap 60/60.

The **honest audit** refits blend weights leave-year-out (fit on 2017 → score 2018, and vice versa):

Candidate	Full-OOF	Honest LOFO	Optimism
champion #59 (37 members)	4.405020	4.437388	+0.032
mega-blend #48	4.406131	4.432727	+0.027
blend #17 (7 members)	4.411229	4.431641	+0.020
seed blend #9 (4 members)	4.415542	4.417713	+0.002
root solo	4.415985	4.415985	0

Every fitted blend loses honestly — the "gains" were OOF-noise fits over near-homogeneous Ridge variants (the one true diversity member, LGB, is 2 SMAPE points worse and earns ~0 honest weight) — so the search's real value here was *vetoing* the blend, not finding gains. A caveat from STATUS.md : with only 2 validation years the LOFO gate is itself high-variance; the conservative solo choice is the point.

The **Loss Function** is L2 on the log target (Ridge), with model selection on SMAPE after inversion and rounding. The **Optimization Algorithm** is a closed-form ridge solve — **not** SGD/ADAM — with (all ultimately vetoed) blend weights searched by Dirichlet sampling plus coordinate-ascent refinement (the harness-v3 default). The **Learning Rate** is *N/A for the champion (closed-form linear solve)*; the LGB member's boosting shrinkage is 0.05. There is **no Learning Rate Scheduler** (*N/A — no iterative gradient training*) and **no Batch Size** (*N/A — full-dataset closed-form fit*).

Training Duration: the full tree search took **91 s of wall-clock for 60 nodes** (the root solo evaluates in 1.4 s); total pipeline wall-clock beyond that is not recorded. **Training Memory** was not recorded — a $26,298 \times 119$ float matrix is trivial on the single arm64 CPU machine. **Transfer Learning** is *N/A (no pretrained weights)*; its analogue is **experience-library transfer** — the structural assumptions from the prior tpsjan22 run were carried in via knowledge/experience.md and *re-confirmed on data* before use, and the dossier's GDP-join prior supplied the highest-leverage feature. **Data Augmentation** is *N/A (tabular)*; the analogues are the two permitted external-data joins (World Bank GDP, holidays calendars).

Reproducibility Standards: the champion is fully deterministic — closed-form Ridge, deterministic expanding-year folds (no fold seed exists to record), and the tree-search root was digit-verified at 4.415985 against the training pipeline before searching. The final submission.csv is a full-train (2015–2018) refit of the exact root config.

Inference procedures

Decision Threshold is *N/A* — SMAPE is a regression forecast metric, so no classification cut-point exists. Post-processing is **round to integer, clip ≥ 1** , applied *inside* the metric during search so every candidate was scored post-processed (a dossier-flagged SMAPE trap). The submission holds 6,570 rows validated against the sample file, predictions in **[100, 2907] with mean 414.0**. **Inference Duration** was not separately profiled — scoring 6,570 rows through a 119-coefficient linear model is effectively instant — and **Inference Memory** is negligible.

Evaluation & Benchmarking

Task for Performance Evaluation. Forecast 2019 daily unit sales for all 18 series. **Performance Metrics.** SMAPE (minimize) is the sole competition metric; our champion reaches **pooled expanding-year CV SMAPE 4.415985** (fold-2017 4.6457 / fold-2018 4.1862), improving 10.806697 $\rightarrow$ 7.021504 $\rightarrow$ 4.415985 down the baseline ladder, with the tree search's 4.40502 rejected by the honest gate. Per the set's standing convention, **between-agent claims in this report use the leaderboard numbers; each agent's local CV describes only its own pipeline's internal view** (the three local figures below were produced under different internal schemes and are not directly comparable to each other).

Performance Benchmarking. The frozen three-way score table compares our agent against the NVIDIA reproduce-agent (copies the strongest public kernel) and the open-source AIDE agent:

Agent	Local SMAPE	Public LB	Private LB	Percentile
Our agent (from-scratch structural Ridge, honest-gated solo)	4.415985	4.90883	6.46156	71.5
NVIDIA reproduce-agent	4.49549	4.43958	4.63651	76.4
AIDE	6.1551	10.11598	10.66638	28.9

We are the **local winner** (`local_winner` = mine) but **NVIDIA wins the leaderboard** (`lb_winner` = nvidia): its Private 4.63651 (percentile 76.4) beats our Private 6.46156 (rank 455/1592, percentile 71.5), while AIDE — whose 6.1551 local score matches the dossier's "official-data-only pipeline" reference point — lands at percentile 28.9. The honest reading of our gap: the LOFO gate correctly killed +0.02–0.03 of blend optimism *within* the training years, but no in-sample gate can guard the 2019 level itself — CV 4.415985 degraded to Public 4.90883 and further to Private 6.46156, a widening drift consistent with the extrapolated GDP + linear-year level going stale across the test year (the public/private split composition is not recorded, so that mechanism is a hypothesis, not a measurement), whereas NVIDIA's copied recipe transferred far better (local 4.49549 $\rightarrow$ Private 4.63651). The episode's lesson pairs with s3e19: task-correct honest CV is necessary — it put us ~4 SMAPE points ahead of AIDE and made the right solo-vs-blend call — but on a pure future-year task it is not sufficient to beat a battle-tested public recipe.

Fields marked "not recorded": total pipeline wall-clock and training peak memory; inference duration and memory (not separately profiled); the public/private split composition of the 2019 test year; NVIDIA and AIDE pipeline internals (outside this workspace — only their frozen scores are cited).

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions were re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the Stage 0.5 problem dossier** (dossier.json written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run were produced by the separate credentialed operator scoring step and are recorded in `benchmark_results/rerun_three_way.csv` ; the figures below revise nothing in the sections above.

- **Lane**: ran and was scored.
- re-run local CV (lane `STATUS.md`): **SMAPE 4.82934** (3-fold expanding-window leave-one-year-out; per fold 2016 5.25121 / 2017 4.75529 / 2018 4.48036), a 5-member weighted blend of regularized CatBoost models on `log(num_sold) - log(GDP per capita)` with the annual level extrapolated analytically.
- Public / private leaderboard for this re-run (`benchmark_results/rerun_three_way.csv`): **Public 4.58859 / Private 6.29895**; `winner_priv` = **nvidia** (NVIDIA 4.63651, AIDE 10.66638).